

Data Structures

Lecture 3: Algorithm Analysis

Soobin Um

Mar. 11, 2026

Outline

- **Recap (복습)**
- **Algorithm**
 - Definition
 - Representation
- **Performance Analysis of Algorithm**
 - Performance Analysis
 - Asymptotic Analysis

Recap

자료구조 학습목표

- 자료구조마다 **비용(cost)**과 **이득(benefit)**이 서로 다를 수 있다.
- 비용(Cost): 자료구조나 프로그램 등이 소비하는 자원의 양
- 자원(Resource): Time, Space
- 자주 사용되는 자료구조를 배운다.
- 자료구조나 프로그램의 비용을 측정하는 법을 이해한다.
 - 새로운 자료구조나 프로그램의 이점을 판단할 수 있다.

추상 자료형(Abstract Data Types)

- 추상자료형 (ADT; Abstract Data Type):

- 자료형(data type)을 값과 연산으로 명세(specification)한 것

- ADT 연산은 입력과 출력으로 정의됨

- Encapsulation: 연산의 구현 방법은 숨긴다

“이 자료형으로 무엇을 할 수 있는지” 만 정의

예시)

- 스택(push, pop 연산 정의 포함)

자료구조(Data Structure)

- 자료구조 (Data Structure): ADT를 실제적으로 구현(physical implementation)한 것.

- main memory에 자료가 어떻게 저장되나?
- 연산은 어떤 구체적인 방법으로 동작하나?

“이 자료형을 어떻게 구현할 것인지” 까지 정의

예시)

- 배열 기반 스택
- 연결리스트 기반 스택

ADT, 자료구조 예시

- Java에서는 ADT를 Interface로 표현
(Python의 경우 추상클래스)
- 자료구조는 Class로 표현

```
interface Dog{
    public void bark();
}

class KoreanDog implements Dog{
    public void bark(){

        System.out.println("멍멍!");
    }
}
```

Java

```
from abc import *
class Dog(metaclass=ABCMeta):
    @abstractmethod
    def bark(self):
        pass

class KoreanDog(Dog):
    def bark(self):
        print("멍멍")
```

Python

Problem, Algorithm, Program

- **문제(Problem):** 풀어야 할 작업
 - 예) 수많은 웹 문서 데이터에서 "Data Mining" 이라는 단어를 포함하는 문서 모두 찾기
- **알고리즘(Algorithm):** 문제를 풀기 위한 방법(method)이나 과정(process)
 - 특정 문제를 푸는 알고리즘은 여러 개 존재 가능
- **프로그램(Program):** 프로그래밍 언어로 알고리즘을 구현한 것

Algorithm

Algorithm

- **알고리즘**

- 어떤 문제를 해결하기 위한 절차나 방법을 공식화한 형태로 표현한 것

- **좋은 알고리즘의 조건**

- 입력: 외부로부터 하나 이상의 입력값을 받아들일 수 있어야 함
- 출력: 최소 하나 이상의 결과를 산출해야 함
- 명확성: 각 단계는 애매하지 않고 명확하게 기술되어야 함
- 유한성: 유한한 단계 안에 반드시 종료되어야 함
- 효율성: 각 단계는 실행 가능한 연산이어야 함

Ways to Represent Algorithms

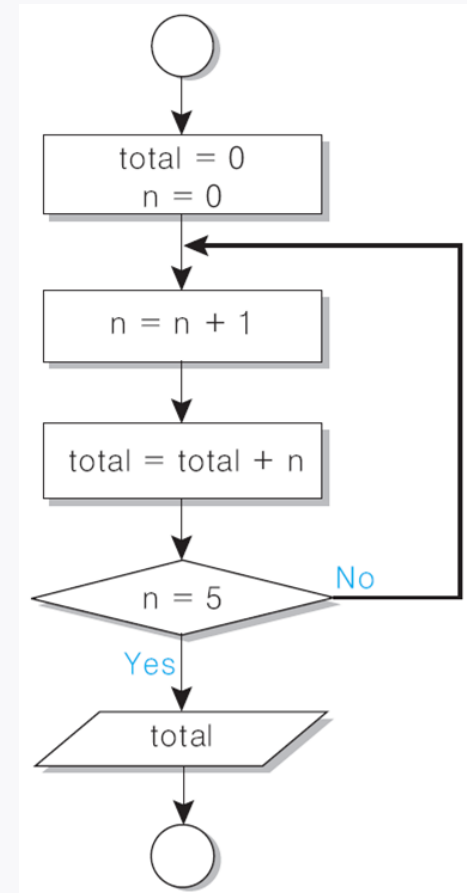
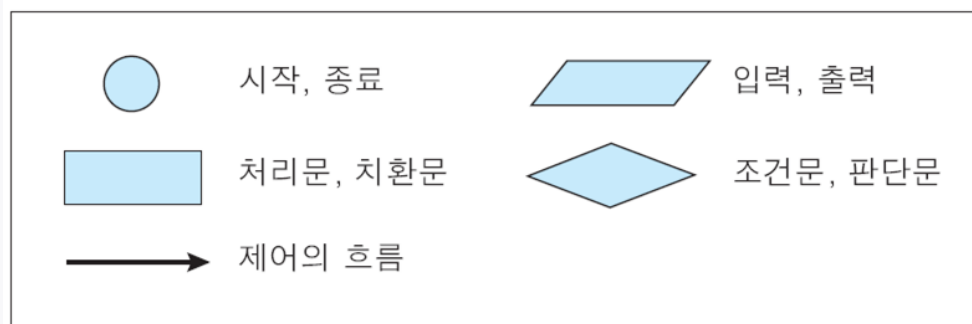
- **자연어 (natural language)**

- 일상적인 언어로 단계들을 서술하는 방식
- 장점: 직관적이고 이해하기 쉬움
- 단점: 모호성이 생길 수 있음
- 예: 선형 탐색
 - 배열에서 원하는 값을 찾을 때, 첫 번째 원소부터 차례대로 비교한다.
 - 만약 원하는 값이 나오면 그 위치를 반환하고, 끝까지 찾지 못하면 실패를 반환한다.

Ways to Represent Algorithms

- 순서도 (flow chart)

- 기호를 이용해 알고리즘의 흐름을 시각적으로 표현
- 장점: 전체 구조와 흐름을 한눈에 파악 가능
- 단점: 복잡한 알고리즘의 경우, 표현이 어려울 수 있음



정수 1부터 5까지의 합을 계산하는 순서도 예

Ways to Represent Algorithms

- **의사코드 (pseudo-code)**

- 실제 프로그래밍 언어와 유사한 형식으로, 언어에 구애받지 않고 작성
- 장점: 실제 구현에 가깝고 명확성 확보 가능
- 단점: 완전히 표준화된 문법이 없음

```
fibonacci(n)
  if (n < 0) then
    stop
  if (n ≤ 1) then
    return n
  f1 ← 0
  f2 ← 1
  for (i in {2, 3, ..., n}) do
    fn ← f1 + f2
    f1 ← f2
    f2 ← fn
  return fn
```

피보나치 수열의 n 번째 항을 구하는 의사코드 예

Ways to Represent Algorithms

- 프로그래밍 언어 (programming language)
 - 실제 프로그래밍 언어로 구현
 - 장점: 실행 가능하고 테스트 가능
 - 단점: 언어 문법에 따라 가독성이 떨어질 수 있음

```
def factorial(n):  
    if n==0:  
        return 1  
    else:  
        return n*factorial(n-1)
```

Python으로 구현된 팩토리얼 함수

Goals of Designing Algorithms

- **이해하기 쉽고, 구현하기 쉽고, 디버깅하기 쉬운 알고리즘 설계**
 - 프로그램을 혼자서만 사용하는 경우는 드뭄. 협업/유지보수 고려가 필수적
 - 읽기 어렵고 구조가 나쁜 코드는 유지보수 비용을 크게 증가시킴
- **컴퓨터 자원을 효율적으로 사용하는 (효율성 높은) 알고리즘 설계**
 - CPU, 메모리, 저장장치 등의 자원은 한정적임
 - 같은 문제라도 자료구조 선택에 따라 실행 시간이 수십 배까지 차이가 날 수 있음



Performance Analysis of Algorithm

Performance Analysis

- 실험적 분석 (empirical analysis)
 - 알고리즘을 실제로 구현 후, 다양한 입력에 대해 실행 시간을 측정

Method	cFID ↓	sFID ↓	Complexity		
			Infer ↓	Extra ↓	Memory ↓
ADM (Dhariwal & Nichol, 2021)	18.37	5.39	0.99 s	–	85.73 MB
EDM (Karras et al., 2022)	19.09	4.73	1.87 s	–	84.48 MB
Sehwag et al. (2022)	11.37	4.69	2.75 s	> 16 d	187.04 MB
Um et al. (2023)	12.47	3.13	2.27 s	> 16 d	184.40 MB
Um & Ye (2024b)	11.24	3.17	2.80 s	–	386.75 MB
Temperature sampling	16.19	4.20	0.99 s	–	85.73 MB
Boost-and-Skip (ours)	12.68	3.18	0.98 s	–	<u>85.73</u> MB

Table 3: **Comparison of complexity across existing samplers on ImageNet 64×64 .** “Infer” represents the inference time measured in seconds/sample. “Extra” refers to the additional time, in **days**, required to construct external classifiers used for guided sampling (Sehwag et al., 2022; Um et al., 2023). “Memory” indicates the peak memory usage, measured in MB/sample. All measurements are based on a single NVIDIA A100 GPU.

실행 시간에 대한 실험적 분석 예*

Performance Analysis

• 이론적 분석 (theoretical analysis)

- 알고리즘을 직접 실행하지 않고, 수학적으로 기본 연산의 수행 횟수를 계산
- 입력 크기 n 에 따라 어떻게 성능이 변하는지 예측 가능

Algorithm 1 MinorityPrompt

Require: $\epsilon_\theta, \mathcal{T}, \mathcal{D}, v_T^{(0)}, \mathcal{P}_S, \mathcal{C}, N, K, w, T, s, \lambda$.

- 1: $z_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 2: **for** $t \leftarrow T$ to 1 **do**
- 3: $\mathcal{C}_{v_t^*} \leftarrow \mathcal{C}$
- 4: **if** $t \bmod N = 0$ **then**
- 5: $v_t^* \leftarrow \text{OPTIMIZEEMB}(z_t, v_t^{(0)}, \epsilon_\theta, \mathcal{T}, K, s, \lambda)$
- 6: $\mathcal{C}_{v_t^*} \leftarrow \mathcal{T}(\mathcal{P}_S; v_t^*)$
- 7: **end if**
- 8: $\tilde{\epsilon}_\theta^w \leftarrow w\epsilon_\theta(z_t, \mathcal{C}_{v_t^*}) + (1 - w)\epsilon_\theta(z_t)$
- 9: $\hat{z}_0^w \leftarrow (z_t - \sqrt{1 - \alpha_t}\tilde{\epsilon}_\theta^w)/\sqrt{\alpha_t}$
- 10: $z_{t-1} \leftarrow \sqrt{\alpha_{t-1}}\hat{z}_0^w + \sqrt{1 - \alpha_{t-1}}\tilde{\epsilon}_\theta^w$
- 11: $v_{t-1}^{(0)} \leftarrow v_t^*$
- 12: **end for**
- 13: **return** $x_0 \leftarrow \mathcal{D}(z_0)$

Algorithm 2 Prompt optimization

- 1: **function** OPTIMIZEEMB($z_t, v_t^{(0)}, \epsilon_\theta, \mathcal{T}, K, s, \lambda$)
- 2: **for** $k \leftarrow 1$ to K **do**
- 3: $\mathcal{C}_v \leftarrow \mathcal{T}(\mathcal{P}_S; v_t^{(k-1)})$
- 4: $\epsilon_\theta^1 \leftarrow \epsilon_\theta(z_t, \mathcal{C}_v)$
- 5: $\hat{z}_0^1 \leftarrow (z_t - \sqrt{1 - \alpha_t}\epsilon_\theta^1)/\sqrt{\alpha_t}$
- 6: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 7: $z_{s|t,0} \leftarrow \sqrt{\alpha_s}\hat{z}_0^1 + \sqrt{1 - \alpha_s}\epsilon$
- 8: $\epsilon_\theta^2 \leftarrow \epsilon_\theta(z_{s|t,0}, \mathcal{C})$
- 9: $\hat{z}_0^2 \leftarrow (z_{s|t,0} - \sqrt{1 - \alpha_s}\epsilon_\theta^2)/\sqrt{\alpha_s}$
- 10: $\mathcal{J}_t \leftarrow \|\hat{z}_0^1 - \text{sg}(\hat{z}_0^2)\|_2^2 + \lambda\|\text{sg}(\hat{z}_0^1) - \hat{z}_0^2\|_2^2$
- 11: $v_t^{(k)} \leftarrow v_t^{(k-1)} + \text{AdamGrad}(\mathcal{J}_t)$
- 12: **end for**
- 13: **return** $v_t^* \leftarrow v_t^{(K)}$
- 14: **end function**

Theoretical Analysis

- **수행 시간 함수 $T(n)$**

- 알고리즘의 수행 시간은 일반적으로 입력 크기 n 의 함수로 표현 가능

$$T(n) = \text{기본 연산의 실행 횟수}$$

- **기본 연산 (basic operation)**

- 알고리즘의 수행 시간을 결정하는 핵심 연산
- 수행 시간이 고정된 연산을 가정:
 - 단순 할당 (assignment)
 - 산술 연산 (+, -, $\times$, $\div$)
 - 비교 연산 (<, >, =)
 - 배열 원소 접근
- 반복문: (내부 기본 연산 횟수 $\times$ 반복 횟수)로 계산

Example: 기본 연산 실행 횟수 계산

- 피보나치 수열의 n 번째 항을 구하는 알고리즘

fibonacci(n)	$n < 0$	$n = 0$	$n = 1$	$n > 1$
01 if (n<0) then 02 stop 03 if (n≤1) then 04 return n 05 f1 ← 0 06 f2 ← 1 07 for (i in {2, 3, ..., n}) do 08 fn ← f1 + f2 09 f1 ← f2 10 f2 ← fn 11 return fn				
T(n) =				

Example: 기본 연산 실행 횟수 계산

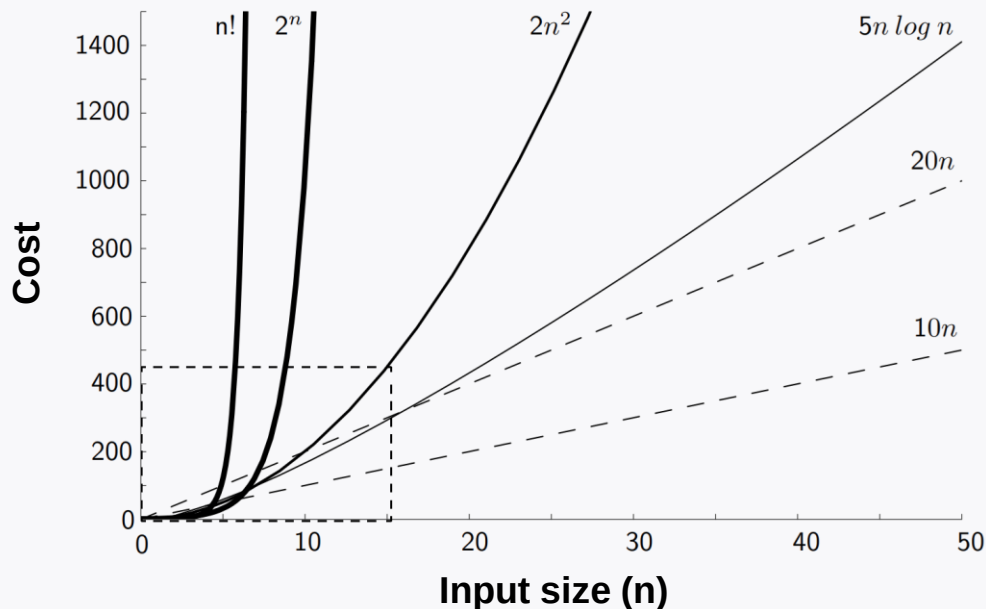
- 피보나치 수열의 n 번째 항을 구하는 알고리즘

fibonacci(n)	$n < 0$	$n = 0$	$n = 1$	$n > 1$
01 if (n<0) then	1	1	1	1
02 stop	1	0	0	0
03 if (n≤1) then	0	1	1	1
04 return n	0	1	1	0
05 f1 ← 0	0	0	0	1
06 f2 ← 1	0	0	0	1
07 for (i in {2, 3, ..., n}) do	0	0	0	n-1
08 fn ← f1 + f2	0	0	0	n-1
09 f1 ← f2	0	0	0	n-1
10 f2 ← fn	0	0	0	n-1
11 return fn	0	0	0	1
T(n) =	2	3	3	4n+1

Growth Rates Comparison

- 각 수행 시간 함수의 n값 변화에 따른 실행 빈도수 비교

logn	<	n	<	nlogn	<	n²	<	n³	<	2ⁿ
0		1		0		1		1		2
1		2		2		4		8		4
2		4		8		16		64		16
3		8		24		64		512		256
4		16		64		256		4096		65536
5		32		160		1024		32768		4294967296



Asymptotic Analysis

- 점근적 분석법 개념

- 입력 크기 n 이 커질수록, 실행 시간이 어떤 속도로 증가하는가를 분석
- n 이 아주 클 때의 성능 예측 가능
- 정확한 실행 시간 계산보다는 증가 추세 파악에 초점

- 점근적 분석법이 필요한 이유

- 일반적으로 **정확한 실제 수행 시간 $T(n)$ 을 구하는 것은 쉽지 않음**
 - 입력 데이터가 복잡한 구조를 가질 수 있음
 - 연산 횟수가 데이터 값이나 분포에 따라 달라질 수 있음
- 큰 입력에서의 성능을 일반화해서 바라보는 것이 더 효과적

Asymptotic Notations

Big-O $O(f(n))$

실행 시간이 최대 어느 정도까지 걸리는지 (상한)

"이보다 더 느리게 동작하지는 않는다"

Big-Ω $\Omega(f(n))$

실행 시간이 최소 어느 정도는 걸린다는 것 (하한)

"아무리 잘 되어도 이보다 빠르지는 않다"

Big-Θ $\Theta(f(n))$

실행 시간이 $f(n)$ 과 같은 차수로 증가 (상/하한 동시)

"실행 시간의 정확한 차수를 전달"

Look ahead

Asymptotic Analysis